

Agent Planning Sheet — [Use case name]

CONTENTS

4	[Use case name]	1
4.1	Plan Your Agent	2
4.2	Build Your Agent Components	5
4.3	Try It Yourself	8
4.4	Expected Output	9

SECTION 4

Plan and Create Your Agent: [Use case name]

This sheet plans and builds an agent for [use case name]. 4.1 plans the agent, 4.2 builds its components, 4.3 puts them together — first in Microsoft 365 Copilot Agent Builder, then on the Copilot Studio GitHub Copilot harness — and 4.4 shows what a correct run looks like.

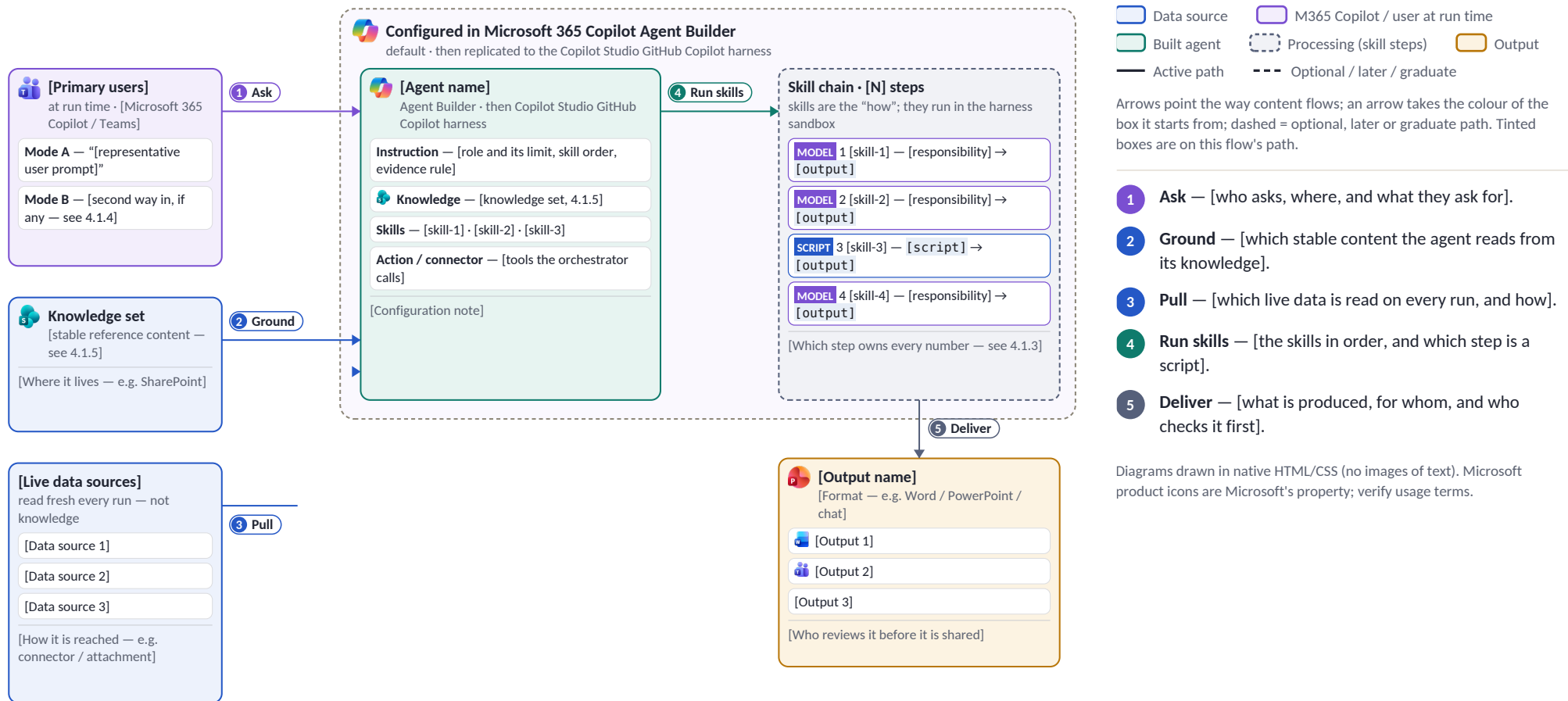
Plan Your Agent

4.1.1 • High level flow

[One or two sentences: what goes in, what the agent does with it, and what comes out.]

High level flow • [Agent name]

Built in: **Microsoft 365 Copilot Agent Builder** • then replicated to **Copilot Studio — GitHub Copilot harness**



4.1.2 • Considerations

WHAT A MODEL-ONLY ANSWER GETS WRONG FOR THIS USE CASE

CONSIDERATION	WHAT HAPPENS	WHY IT IS STRUCTURAL
[Consideration 1 — e.g. Completeness]	[What goes wrong if the model is left to do it]	[Why better prompting will not fix it]
[Consideration 2 — e.g. Arithmetic]	[What goes wrong if the model is left to do it]	[Why better prompting will not fix it]
[Consideration 3 — e.g. Consistency]	[What goes wrong if the model is left to do it]	[Why better prompting will not fix it]
[Consideration 4 — e.g. Traceability]	[What goes wrong if the model is left to do it]	[Why better prompting will not fix it]

The reframe that makes this buildable: [which part of the problem is deterministic, which part needs judgement, and why separating them is the design].

4.1.3 • The determinism split

WHO DOES WHAT, AND WHY

TASK	OWNER	REASONING
[Read the sources]	Orchestrator + tools	[Why the orchestrator owns it]
[Normalise or map the data]	Rule / mapping	[Why a rule, not a guess]
[Calculation or check with one right answer]	Script	[Why a script]
[Calculation or check with one right answer]	Script	[Why a script]
[Interpretation that needs judgement]	Model	[Why the model — and what it must cite]
[Narrative / summary for the audience]	Model	[Why the model]
[Render the output file]	Harness + template	[Why a template keeps the layout identical every run]

THE RULE OF THUMB

If two experts looking at the same data would reach the *identical* answer, it belongs in a script. If they might reasonably differ, it belongs to the model, with the evidence quoted so the disagreement stays visible.

4.1.4 • What the agent handles — run modes

SAME SKILLS, DIFFERENT WAYS IN

	A — [MODE NAME]	B — [MODE NAME]
The question	"[Typical prompt]"	"[Typical prompt]"
Who is present	[Who asks and what they confirm]	[Who asks and what they confirm]
Output	[What this mode produces]	[What this mode produces]
[Other difference]	[Value]	[Value]

4.1.5 • The knowledge set

WHAT THE AGENT IS GROUNDED ON

KNOWLEDGE SOURCE	WHAT IT CONTAINS	WHERE IT LIVES	OWNER
[Knowledge source 1]	[What the agent uses it for]	[e.g. SharePoint site / library / file]	[Owner]
[Knowledge source 2]	[What the agent uses it for]	[e.g. SharePoint site / library / file]	[Owner]
[Knowledge source 3]	[What the agent uses it for]	[e.g. SharePoint site / library / file]	[Owner]

Keep knowledge **thin and stable** — reference content that changes rarely. Data that changes every run is not knowledge; it is read fresh by tools or attached per run: [list the live data, if any].

4.1.6 • The skill set

PROPOSED SKILLS AND THEIR CONTRACTS

SKILL	RESPONSIBILITY	PRODUCES	WEIGHT
[skill-1]	[One responsibility]	[named output]	[Instructions only]
[skill-2]	[One responsibility]	[named output]	[Instructions + resources]
[skill-3]	[One responsibility]	[named output]	[Script + rules]
[skill-4]	[One responsibility]	[named output]	[Instructions + template]

Every contract is a **named output with a schema**, not “help with [topic]”. That is what makes the chain inspectable: each skill reads the previous skill's output rather than a paraphrase of it.

Build Your Agent Components

Everything [Agent name] is made of, component by component. 4.3 assembles them.

4.2.1 • Agent name

• AGENT NAME

[Agent name]

4.2.2 • Agent description

• AGENT DESCRIPTION

[What the agent does, for whom, and what it does not do – one or two sentences.]

4.2.3 • List of knowledge

KNOWLEDGE TO ADD TO THE AGENT

#	KNOWLEDGE	TYPE	LOCATION / LINK
1	[Knowledge source 1]	[SharePoint site / file / Work content / web]	[Location or link]
2	[Knowledge source 2]	[SharePoint site / file / Work content / web]	[Location or link]
3	[Knowledge source 3]	[SharePoint site / file / Work content / web]	[Location or link]

4.2.4 • Instructions

With every procedure packaged as a skill, the instruction block carries only what is true on *every* run: the role and its limit, the skill order, where numbers come from, the evidence rule, writing style and boundaries.

• INSTRUCTIONS • [N] CHARACTERS

```
# [Agent name] - agent instructions
You support [users]. You [what the agent does]. You do not [what the
agent must never do].
## Skills and order
Always work in this order.
1. `[skill-1]` - [when it runs and what it produces].
2. `[skill-2]` - [when it runs and what it produces].
3. `[skill-3]` - [when it runs and what it produces].
4. `[skill-4]` - [when it runs and what it produces].
## Numbers and facts
- [Which skill every number comes from. Never compute or change one yourself.]
- [What to do when an input or rule is missing.]
## Evidence
- [Every statement names its source.]
- [What to say when the sources do not answer the question.]
## Writing
- Language: [default language].
- [Structure and length for each audience.]
## Boundaries
- [What the agent must not send, change or decide.]
```

4.2.5 • Skills

Each skill is a folder with a `SKILL.md` — YAML front matter with `name` and `description`, then the procedure. The description is what the orchestrator reads to decide when the skill applies, so it is written as a trigger, not a label.

```
[skill-1]/
└─ SKILL.md
[skill-2]/
└─ SKILL.md
└─ [resources]/           # [e.g. field mappings, reference tables]
[skill-3]/
└─ SKILL.md
└─ rules/[rules].json    # [rules owned by the business]
└─ scripts/[script].py
[skill-4]/
└─ SKILL.md
└─ templates/           # [output template].docx / .pptx
```

Skill 1 — [skill-1]

• [SKILL-1] / SKILL.MD

```
---
name: [skill-1]
description: [What this skill does and when to use it - written as a trigger.]
---
# [skill-1]
[One paragraph: what this skill does, and what it does not do.]
## Steps
1. [Step]
2. [Step]
3. [Step]
## Output - [named output]
[The schema or format of the output the next skill reads.]
## Gate
[What must be true before the next skill runs.]
```

Skill 2 — [skill-2]

• [SKILL-2] / SKILL.MD

```
---
name: [skill-2]
description: [What this skill does and when to use it - written as a trigger.]
---
# [skill-2]
[One paragraph: what this skill does, and what it does not do.]
## Input
[The named output this skill reads.]
## Steps
1. [Step]
2. [Step]
## Output - [named output]
[The schema or format of the output.]
```

Skill 3 — [skill-3] (script + rules)

• [SKILL-3] / SKILL.MD

```
---
name: [skill-3]
description: [What this skill does and when to use it - written as a trigger.]
---
# [skill-3]
[One paragraph: what the script calculates. The model never recalculates
or changes the script's output.]
## Run
python scripts/[script].py --input [input file] --rules rules/[rules].json
## Output - [named output]
[The fields the script returns.]
## If the script fails or a rule is missing
[What to report instead of guessing.]
```

• [SKILL-3] / SCRIPTS/[SCRIPT].PY

[Script – Python standard library only]

• [SKILL-3] / RULES/[RULES].JSON

[Rules / thresholds – owned by the business]

Skill 4 — [skill-4]

• [SKILL-4] / SKILL.MD

```
---
name: [skill-4]
description: [What this skill does and when to use it - written as a trigger.]
---
# [skill-4]
[One paragraph: what this skill produces, for which audience.]
## Input
[The named outputs this skill reads.]
## Structure
1. [Section]
2. [Section]
3. [Section]
## Rules
- [Copy numbers from the earlier output; never recalculate.]
- [Use templates/[output template] for the layout.]
```

4.2.6 • Sample prompts

SAMPLE PROMPTS FOR THE AGENT'S FRONT DOOR

TITLE	MESSAGE
[Prompt title 1]	[Prompt message 1]
[Prompt title 2]	[Prompt message 2]
[Prompt title 3]	[Prompt message 3]

Try It Yourself

Put the components from 4.2 together. Build and test in **Microsoft 365 Copilot Agent Builder** first, then replicate the same components to the **Copilot Studio GitHub Copilot harness**. Test each against the expected output in 4.4.

THE BUILD ARTEFACTS Name and description, instructions, knowledge, [N] skills, sample prompts

Name & description 4.2.1 • 4.2.2

[Agent name] and its description

Agent Builder: **Configure**

Instructions 4.2.4

The instruction block

Agent Builder: **Configure → Instructions**

Knowledge 4.2.3

The list of knowledge

Agent Builder: **Configure → Knowledge**

Skills 4.2.5

[skill-1] · [skill-2] · [skill-3] · [skill-4], each packaged as a **.zip**

Agent Builder: **Configure → Skills**

Sample prompts 4.2.6

Title and message for each prompt

Agent Builder: **Configure → Suggested prompts**

Steps — Microsoft 365 Copilot Agent Builder

1. **Create the agent.** In Microsoft 365 Copilot, open Agent Builder, create a new agent and switch to the *Configure* pane.
2. **Name and describe it.** Enter **[Agent name]** and paste the description from 4.2.2.
3. **Paste the instructions.** *Configure → Instructions*: paste the block from 4.2.4.
4. **Add the skills.** *Configure → Skills → + Add*: upload each skill from 4.2.5 as a complete **.zip** — a bare **SKILL.md** is rejected. Maximum 8 skills per agent. Skills are in preview and need a Premium licence with Frontier access.
5. **Add knowledge.** *Configure → Knowledge → + Add knowledge*: add each source from 4.2.3. Switch on only the *Work content* sources the agent needs (Cloud files, Outlook, Teams, Copilot connectors), and set *Web search* on or off.
6. **Add the sample prompts.** *Configure → Suggested prompts*: add the title and message of each prompt from 4.2.6.
7. **Test with Try it.** Turn on the *Try it* toggle, run each sample prompt and compare the result with 4.4.
8. **Create and share.** Create the agent and share it with your test users.

Steps — replicate to the Copilot Studio GitHub Copilot harness

1. **Create the agent.** From the Copilot Studio home page, create a new agent and choose the **GitHub Copilot harness**. The harness cannot be changed later — decide now. Credits are consumed from the moment you start building.
2. **Name and describe it.** On the *Build* tab, enter the name and description from 4.2.1 and 4.2.2.
3. **Paste the instructions.** *Build → Instructions*: paste the same block from 4.2.4.
4. **Add knowledge.** *Build → Knowledge*: add the sources from 4.2.3.
5. **Add the skills and tools.** *Build → Tools and skills*: upload each skill — a **SKILL.md**, or a packaged **.zip** for skills that carry a script, rules or templates — and connect the tools the agent needs: [connectors / tools].
6. **Choose the model and memory.** *Build → Model*: [model]. *Build → Memory*: [on / off] (preview).
7. **Test in the Preview tab.** Run the sample prompts from 4.2.6 and compare the result with 4.4.
8. **Create a test set in the Evaluate tab.** [Test cases — including at least one missing input and one edge case.]
9. **Publish and monitor.** Publish to [Teams / Microsoft 365 Copilot], then track activity in the *Monitor* tab.

Expected Output

What a correct run looks like — sample output and the acceptance checklist

[What was run, on which test data, as of which date.]

[Sample prompt]

[Sample output – script output or agent reply]

[SAMPLE OUTPUT TABLE]

[COLUMN 1]	[COLUMN 2]	[COLUMN 3]	[COLUMN 4]
[Value]	[Value]	[Value]	[Value]
[Value]	[Value]	[Value]	[Value]
[Value]	[Value]	[Value]	[Value]

WHAT THE OUTPUT MUST CONTAIN — THE ACCEPTANCE CHECKLIST

PART	A CORRECT RUN SHOWS
[Part 1]	[What must be there]
[Part 2]	[What must be there]
[Part 3]	[What must be there]
[Part 4]	[What must be there]

WHAT THIS PROVES

[Which finding appears only because a script or rule did the work — the determinism split earning its place.]